

Obstáculos para uma Lhama

Uma lhama quer viajar pelo Planalto Andino. Ela tem um mapa do planalto na forma de uma grade de $N \times M$ células quadradas. As linhas do mapa são numeradas de 0 a $N - 1$ de cima para baixo, e as colunas são numeradas de 0 a $M - 1$ da esquerda para a direita. A célula do mapa na linha i e coluna j ($0 \leq i < N, 0 \leq j < M$) é denotada por (i, j) .

A lhama estudou o clima do planalto e descobriu que todas as células em cada linha do mapa têm a mesma **temperatura** e todas as células em cada coluna do mapa têm a mesma **umidade**. A lhama forneceu a você dois vetores de inteiros T e H de comprimento N e M , respectivamente. Aqui $T[i]$ ($0 \leq i < N$) indica a temperatura das células na linha i , e $H[j]$ ($0 \leq j < M$) indica a umidade das células na coluna j .

A lhama também estudou a flora do planalto e notou que uma célula (i, j) está **livre de vegetação** se e somente se sua temperatura for maior que sua umidade, formalmente $T[i] > H[j]$.

A lhama só consegue atravessar o planalto seguindo **caminhos válidos**. Um caminho válido é uma sequência de células distintas que satisfazem as seguintes condições:

- Cada par de células consecutivas no caminho compartilha um lado comum.
- Todas as células no caminho estão livres de vegetação.

Sua tarefa é responder a Q perguntas. Para cada pergunta, você recebe quatro números inteiros: L, R, S e D . Você deve determinar se existe um caminho válido tal que:

- O caminho começa na célula $(0, S)$ e termina na célula $(0, D)$.
- Todas as células no caminho estão entre as colunas L a R , inclusive.

É garantido que tanto $(0, S)$ quanto $(0, D)$ estão livres de vegetação.

Detalhes de implementação

O primeiro procedimento que você deve implementar é:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : um vetor de comprimento N especificando a temperatura em cada linha.
- H : um vetor de comprimento M especificando a umidade em cada coluna.

- Este procedimento é chamado exatamente uma vez para cada caso de teste, antes de qualquer chamada para `can_reach`.

O segundo procedimento que você deve implementar é:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : números inteiros que descrevem uma pergunta.
- Este procedimento é chamado Q vezes para cada caso de teste.

Este procedimento deve retornar `true` se e somente se houver um caminho válido da célula $(0, S)$ para a célula $(0, D)$, de modo que todas as células no caminho estejam entre as colunas L a R , inclusive.

Restrições

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ para cada i tal que $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ para cada j tal que $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- Ambas as células $(0, S)$ e $(0, D)$ estão livres de vegetação.

Subtarefas

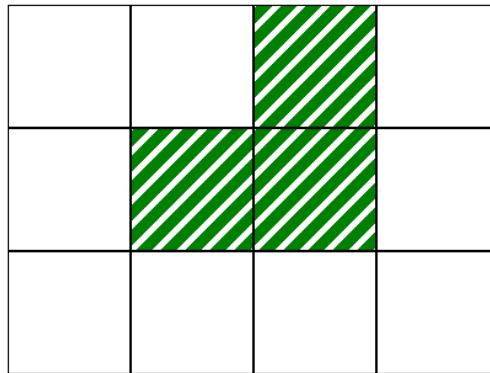
Subtarefa	Pontuação	Restrições adicionais
1	10	$L = 0, R = M - 1$ para cada pergunta. $N = 1$.
2	14	$L = 0, R = M - 1$ para cada pergunta. $T[i - 1] \leq T[i]$ para cada i tal que $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ para cada pergunta. $N = 3$ e $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ para cada pergunta. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ para cada pergunta.
6	17	Sem restrições adicionais.

Exemplo

Considere a seguinte chamada:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

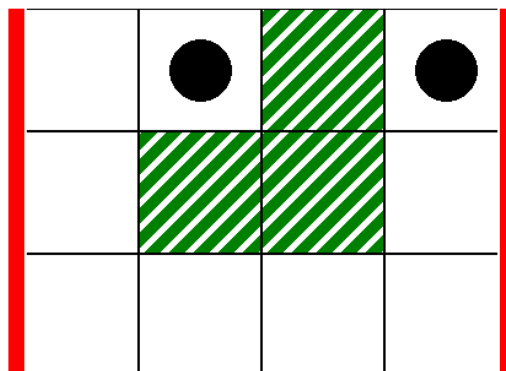
Isso corresponde ao mapa na imagem a seguir, onde as células brancas estão livres de vegetação:



Como primeira pergunta, considere a seguinte chamada:

```
can_reach(0, 3, 1, 3)
```

Isso corresponde ao cenário na imagem a seguir, onde as linhas verticais grossas indicam o intervalo de colunas de $L = 0$ a $R = 3$, e os discos pretos indicam as células inicial e final:



Neste caso, a lhama pode ir da célula $(0, 1)$ até a célula $(0, 3)$ através do seguinte caminho válido:

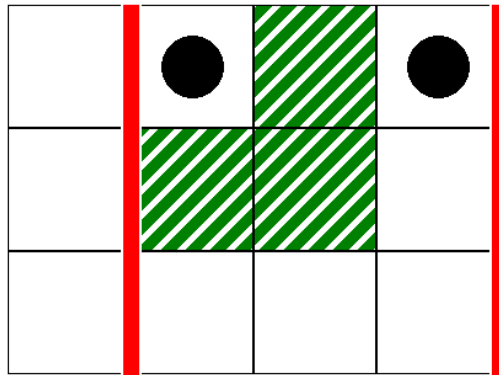
$(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)$

Portanto, esta chamada deve retornar `true`.

Como segunda pergunta, considere a seguinte chamada:

```
can_reach(1, 3, 1, 3)
```

Isso corresponde ao cenário na imagem a seguir:



Neste caso, não há um caminho válido da célula $(0,1)$ para a célula $(0,3)$, de modo que todas as células no caminho estejam entre as colunas 1 a 3, inclusive. Portanto, esta chamada deve retornar `false`.

Corretor de Exemplo

Formato de entrada:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Aqui, $L[k]$, $R[k]$, $S[k]$ e $D[k]$ ($0 \leq k < Q$) especificam os parâmetros para cada chamada para `can_reach`.

Formato de Saída:

```
A[0]
A[1]
...
A[Q-1]
```

Aqui, $A[k]$ ($0 \leq k < Q$) é 1 se a chamada `can_reach(L[k], R[k], S[k], D[k])` retornou `true` e 0 caso contrário.